FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Sagpang, Edriane P.
Section:	AN43
Professor:	Ms. Tan
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

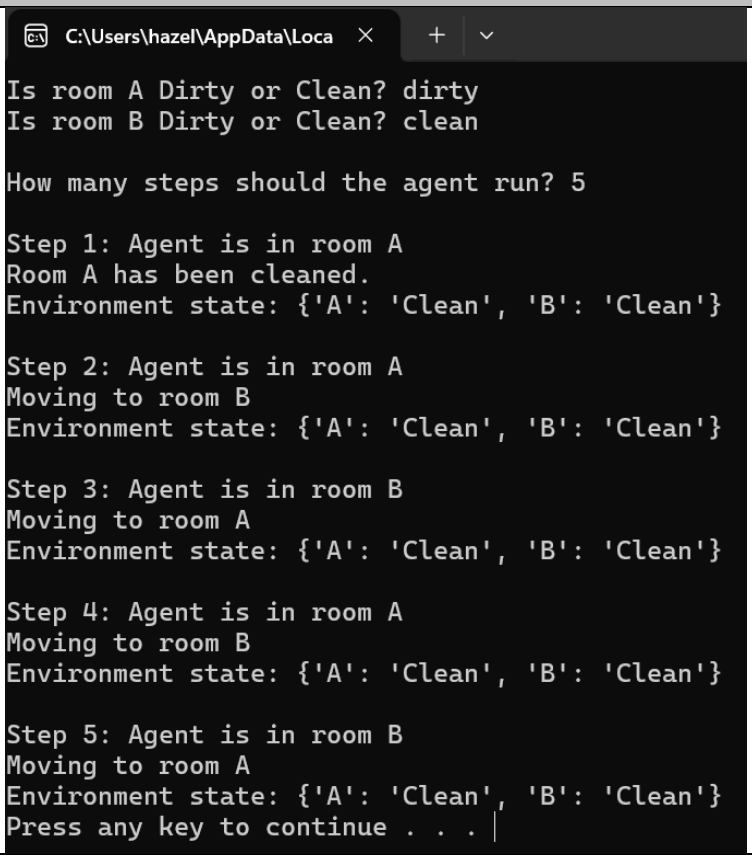
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

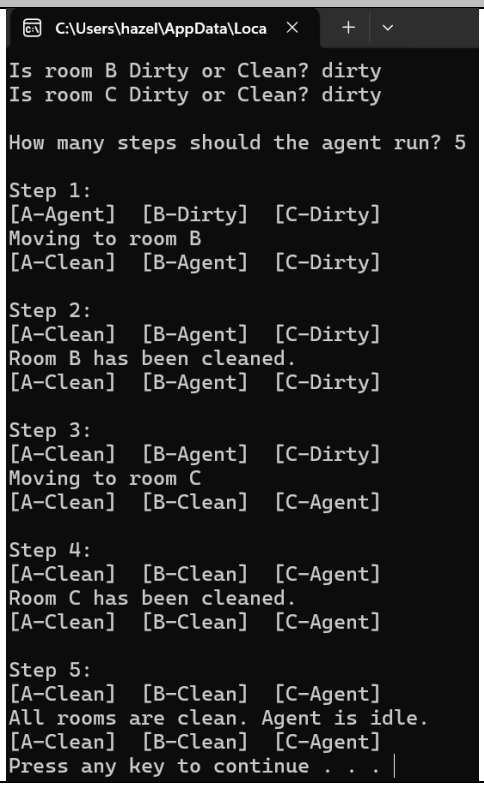
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
   - The agent follows two rules.
     Rule 1: If the room is dirty, clean it.
     Rule 2: If the room is clean, go to the other room.

2. How does the agent decide when to clean and when to move?
   - The agent looks at the room it is standing in.
     If the room is dirty, it cleans the room.
     If the room is clean, it moves to the other room.
     The agent does only one thing each step. It either cleans or moves, never both.

3. What happens if both rooms are already clean at the start?
   - The agent has nothing to clean. So it just keeps moving back and forth between room A and room B. It does this until the steps run out. It does not know it can stop, because it has no rule for “everything is clean.”

4. How would you modify the agent to handle **more than two rooms**?
   - Keep all the rooms in a list, like A, B, C, and D. This way, we are not stuck with only two rooms.
     Change the move rule. Instead of "go to the other room," the agent goes to the next room in the list. Or it picks a dirty room at random.
     Add a stop rule. If no room is dirty, the agent stays still.

Answer the Questions ⇩

```
def ask_room_state(room):
    """Ask the user for the initial state of a room until the input is valid."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if answer in ("dirty", "clean"):
            return answer.capitalize()      # stored as 'Dirty' or 'Clean'
        print("  Please type either 'dirty' or 'clean'.")


def is_dirty(environment, room):
    """Return True if the given room is dirty."""
    return environment[room] == "Dirty"
```

```
def clean_room(environment, room):
    """Clean the given room (update the environment)."""
    environment[room] = "Clean"


# ---------------------------------------------------------------------
# TASK 2: The rule-based agent
# ---------------------------------------------------------------------
class VacuumAgent:
    def __init__(self, start_room="A"):
        self.location = start_room          # the room the agent is in

    def move(self):
        """Switch to the other room (A <-> B)."""
        self.location = "B" if self.location == "A" else "A"

    def perceive_and_act(self, environment):
        """
        Perceive the current room, then apply the rules.
        Returns a text describing the action taken.
        """
        # Perceive: is the current room dirty?
        if is_dirty(environment, self.location):
            # Rule 1: current room is dirty -> clean it
            clean_room(environment, self.location)
            return f"Room {self.location} has been cleaned."
        else:
            # Rule 2: current room is clean -> move to the other room
            self.move()
            return f"Moving to room {self.location}"


# ---------------------------------------------------------------------
# TASK 3: Run the simulation
# ---------------------------------------------------------------------
def ask_steps():
    """Ask the user how many steps to run (must be a whole number >= 1)."""
    while True:
        try:
            steps = int(input("How many steps should the agent run? "))
            if steps >= 1:
                return steps
        except ValueError:
            pass
```

```
        print("  Please enter a whole number of 1 or more.")


def main():
    # Initialize the two rooms using a dictionary
    environment = {"A": None, "B": None}

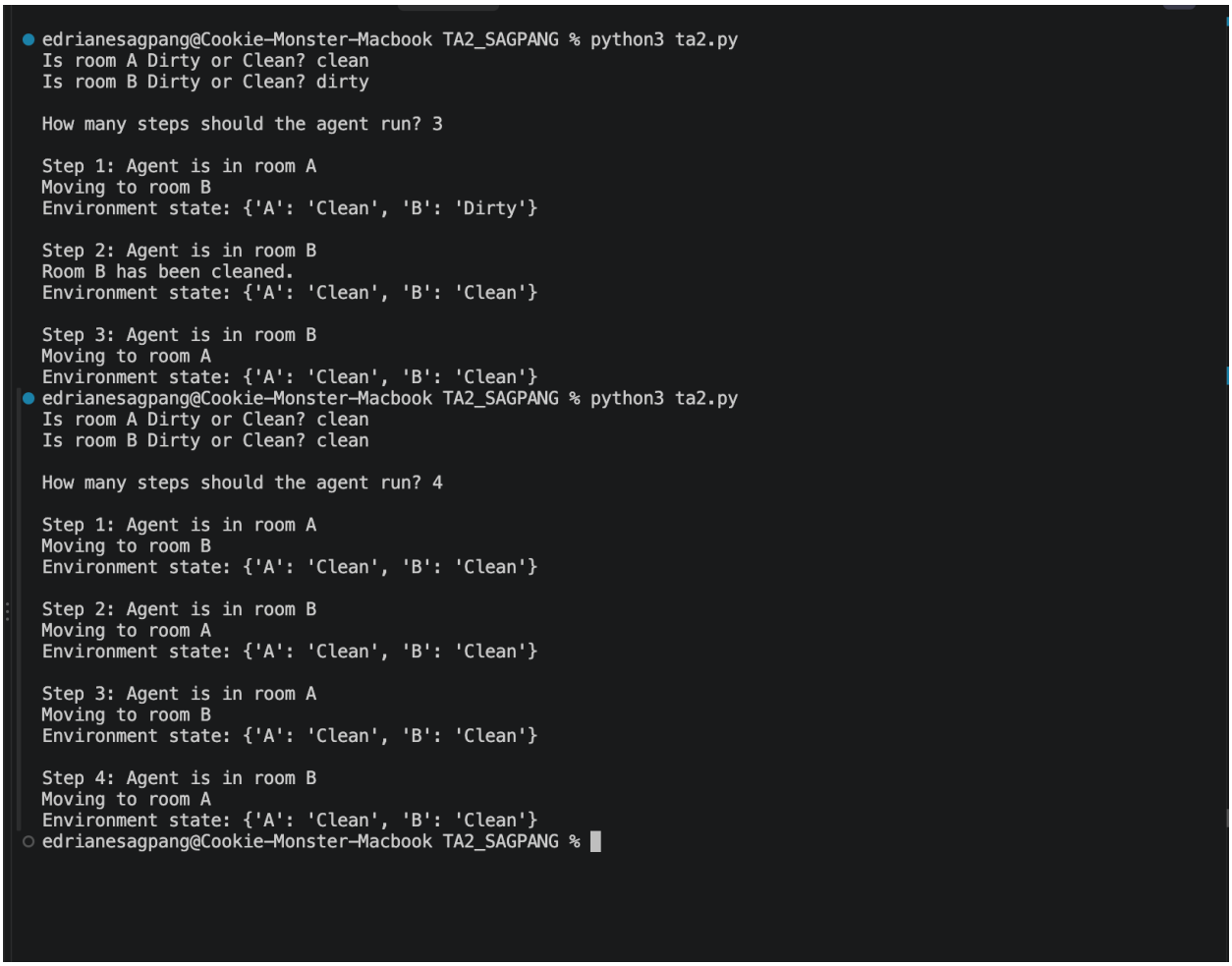
    # Let the user set the initial state of each room
    for room in environment:
        environment[room] = ask_room_state(room)

    print()
    steps = ask_steps()

    agent = VacuumAgent("A")              # agent starts in room A

    # Run the agent in a loop for the chosen number of steps
    for step in range(1, steps + 1):
        print(f"\nStep {step}: Agent is in room {agent.location}")
        action = agent.perceive_and_act(environment)
        print(action)
        print(f"Environment state: {environment}")


if __name__ == "__main__":
    main()
```


edrianesagpang@Cookie-Monster-Macbook TA2_SAGPANG % python3 ta2.py
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
How many steps should the agent run? 3
Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}
Step 2: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
edrianesagpang@Cookie-Monster-Macbook TA2_SAGPANG % python3 ta2.py
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? clean
How many steps should the agent run? 4
Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
edrianesagpang@Cookie-Monster-Macbook TA2_SAGPANG %